VendaService.java

```
package net.originmobi.pdv.service;

import java.sql.Date;
import java.sql.Timestamp;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import net.originmobi.pdv.controller.TituloService;
import net.originmobi.pdv.enumerado.EntradaSaida;
import net.originmobi.pdv.enumerado.TituloTipo;
import net.originmobi.pdv.enumerado.VendaSituacao;
import net.originmobi.pdv.enumerado.caixa.EstiloLancamento;
import net.originmobi.pdv.enumerado.caixa.TipoLancamento;
import net.originmobi.pdv.filter.VendaFilter;
import net.originmobi.pdv.model.Caixa;
import net.originmobi.pdv.model.CaixaLancamento;
import net.originmobi.pdv.model.PagamentoTipo;
import net.originmobi.pdv.model.Receber;
import net.originmobi.pdv.model.Titulo;
import net.originmobi.pdv.model.Usuario;
import net.originmobi.pdv.model.Venda;
import net.originmobi.pdv.model.VendaProduto;
import net.originmobi.pdv.repository.VendaRepository;
import net.originmobi.pdv.service.cartao.CartaoLancamentoService;
import net.originmobi.pdv.singleton.Aplicacao;
import net.originmobi.pdv.utilitarios.DataAtual;

@Service
public class VendaService {

	@Autowired
	private VendaRepository vendas;

	@Autowired
	private UsuarioService usuarios;

	@Autowired
	private VendaProdutoService vendaProdutos;

	@Autowired
	private PagamentoTipoService formaPagamentos;

	@Autowired
	private CaixaService caixas;

	@Autowired
	private ReceberService receberServ;

	@Autowired
	private ParcelaService parcelas;

	@Autowired
	private CaixaLancamentoService lancamentos;
```

```
	@Autowired
	private TituloService tituloService;

	@Autowired
	private CartaoLancamentoService cartaoLancamento;

	@Autowired
	private ProdutoService produtos;

	private Timestamp dataHoraAtual = new Timestamp(System.currentTimeMillis());

	public Long abreVenda(Venda venda) {
		if (venda.getCodigo() == null) {
			Aplicacao aplicacao = Aplicacao.getInstancia();
			Usuario usuario =
usuarios.buscaUsuario(aplicacao.getUsuarioAtual());

			venda.setData_cadastro(dataHoraAtual);
			venda.setSituacao(VendaSituacao.ABERTA);
			venda.setUsuario(usuario);
			venda.setValor_produtos(0.00);

			try {
				vendas.save(venda);
			} catch (Exception e) {
				e.getStackTrace();
			}

		} else {

			try {
				vendas.updateDadosVenda(venda.getPessoa(),
venda.getObservacao(), venda.getCodigo());
			} catch (Exception e) {
				e.getStackTrace();
			}

		}

		return venda.getCodigo();
	}

	public Page<Venda> busca(VendaFilter filter, String situacao, Pageable
pageable) {

		VendaSituacao situacaoVenda = situacao.equals("ABERTA") ?
VendaSituacao.ABERTA : VendaSituacao.FECHADA;

		if (filter.getCodigo() != null)
			return vendas.findByCodigoIn(filter.getCodigo(), pageable);
		else
			return vendas.findBySituacaoEquals(situacaoVenda, pageable);
	}

	public String addProduto(Long codVen, Long codPro, Double vlBalanca) {
		String vendaSituacao = vendas.verificaSituacao(codVen);

		if (vendaSituacao.equals(VendaSituacao.ABERTA.toString())) {
			VendaProduto vendaProduto = null;

			vendaProduto = new VendaProduto(codPro, codVen, vlBalanca);

			try {
				vendaProdutos.salvar(vendaProduto);
			} catch (Exception e) {
				e.getStackTrace();
			}
```

```
                } else {
                        return "Venda fechada";
                }

                return "ok";
        }

        public String removeProduto(Long posicaoProd, Long codVenda) {
                try {
                        Venda venda = vendas.findByCodigoEquals(codVenda);
                        if (venda.getSituacao().equals(VendaSituacao.ABERTA))
                                vendaProdutos.removeProduto(posicaoProd);
                        else
                                return "Venda fechada";
                } catch (Exception e) {
                        e.getStackTrace();
                }

                return "ok";
        }

        public List<Venda> lista() {
                return vendas.findAll();
        }

        @Transactional(readOnly = false, propagation = Propagation.REQUIRED)
        public String fechaVenda(Long venda, Long pagamentotipo, Double vlprodutos,
Double desconto, Double acrescimo,
                        String[] vlParcelas, String[] titulos) {

                if (!vendaIsAberta(venda))
                        throw new RuntimeException("venda fechada");

                if (vlprodutos <= 0)
                        throw new RuntimeException("Venda sem valor, verifique");

                DataAtual dataAtual = new DataAtual();
                PagamentoTipo formaPagamento = formaPagamentos.busca(pagamentotipo);

                String[] formaPagar = formaPagamento.getFormaPagamento().replace("/",
" ").split(" ");

                // vlTotal é usado no lancamento
                Double vlTotal = (vlprodutos + acrescimo) - desconto;

                int qtdVezes = formaPagar.length;

                int sequencia = 1;

                // crio este MAP para receber todas as formas de pagamento e poder
                // pesquisar por uma chave que é o tipo da forma, assim posso
verificar
                // se o tipo 00 esta presente.
                Map<String, String> modeloPagar = new HashMap<>();
                for (int i = 0; i < formaPagar.length; i++) {
                        modeloPagar.put(formaPagar[i], formaPagar[i]);
                }

                Venda dadosVenda = vendas.findByCodigoEquals(venda);
                dadosVenda.setPagamentotipo(formaPagamento);

                // gera um receber
                Receber receber = new Receber("Recebimento referente a venda " +
venda, vlTotal, dadosVenda.getPessoa(),
                                dataAtual.dataAtualTimeStamp(), dadosVenda);

                try {
                        receberServ.cadastrar(receber);
                } catch (Exception e) {
```

```
                        System.out.println(e);
                        throw new RuntimeException("Erro ao fechar a venda, chame o
suporte");
                }

                Double desc = desconto / vlParcelas.length;
                Double acre = acrescimo / vlParcelas.length;

                // verifica a forma de pagamento para realizar o lançamento
apropriado
                for (int i = 0; i < formaPagar.length; i++) {
                        Optional<Titulo> titulo =
tituloService.busca(Long.decode(titulos[i]));

                        // venda à vista
                        if (formaPagar[i].equals("00")) {

                                // no dinheiro
                                if
(titulo.get().getTipo().getSigla().equals(TituloTipo.DIN.toString())) {
                                        // verifica se o caixa esta aberto para
realizar o lançamento no mesmo
                                        if (!caixas.caixaIsAberto())
                                                throw new RuntimeException("nenhum
caixa aberto");

                                        qtdVezes = avistaDinheiro(vlprodutos,
vlParcelas, formaPagar, qtdVezes, i, desc, acre);
                                }

                                // se for no cartão de debito ou crédito
                                else if
(titulo.get().getTipo().getSigla().equals(TituloTipo.CARTDEB.toString())
                                                ||
titulo.get().getTipo().getSigla().equals(TituloTipo.CARTCRED.toString())) {

                                        Double vl_parcela =
Double.valueOf(vlParcelas[i]);

                                        cartaoLancamento.lancamento(vl_parcela,
titulo);
                                }

                        } else {
                                // venda a prazo

                                if (dadosVenda.getPessoa() == null)
                                        throw new RuntimeException("Venda sem
cliente, verifique");

                                // no dinheiro
                                sequencia = aprazo(vlprodutos, vlParcelas, dataAtual,
formaPagar, qtdVezes, sequencia, receber, i, desc,
                                                acre);
                        }

                        try {
                                Double vlFinal = (vlprodutos + acrescimo) - desconto;
                                // realiza o fechamento da venda
                                vendas.fechaVenda(venda, VendaSituacao.FECHADA,
vlFinal, desconto, acrescimo,
                                                dataAtual.dataAtualTimeStamp(),
formaPagamento);
                        } catch (Exception e) {
                                System.out.println(e);
                                throw new RuntimeException("Erro ao fechar a venda,
chame o suporte");
                        }
```

```
                }

                // Responsável por realizar a movimentação de estoque
                produtos.movimentaEstoque(venda, EntradaSaida.SAIDA);

                return "Venda finalizada com sucesso";
        }

        /*
         * Responsável por realizar o lançamento quando a parcela da venda é a prazo
         *
         */
        private int aprazo(Double vlprodutos, String[] vlParcelas, DataAtual
dataAtual, String[] formaPagar, int qtdVezes,
                        int sequencia, Receber receber, int i, Double acre, Double
desc) {

                if (vlParcelas[i].isEmpty()) {
                        throw new RuntimeException("valor de recebimento invalido");
                }

                try {
                        Double valor_parcela = (Double.valueOf(vlParcelas[i]) + acre)
- desc;
                        parcelas.gerarParcela(valor_parcela, 0.00, 0.00, 0.0,
valor_parcela, receber, 0, sequencia,
                                        dataAtual.dataAtualTimeStamp(),

Date.valueOf(dataAtual.DataAtualIncrementa(Integer.parseInt(formaPagar[i]))));

                } catch (Exception e) {
                        e.getMessage();
                        throw new RuntimeException();
                }

                sequencia++;
                return sequencia;
        }

        /*
         * Responsável por realizar o lançamento quando a parcela da venda é à vista
e
         * no dinheiro
         *
         */
        private int avistaDinheiro(Double vlprodutos, String[] vlParcelas, String[]
formaPagar, int qtdVezes, int i,
                        Double acre, Double desc) {

                // decremento ela para usa-la no a prazo, sem a sequencia do a
                // vista
                qtdVezes = qtdVezes - 1;

                if (vlParcelas[i].isEmpty())
                        throw new RuntimeException("Parcela sem valor, verifique");

                Double totalParcelas = 0.0;

                // pega a soma de todas as parcelas para comparar com o valor
recebido
                for (int aux = 0; aux < vlParcelas.length; aux++)
                        totalParcelas += Double.valueOf(vlParcelas[i]);

                if (!totalParcelas.equals(vlprodutos))
                        throw new RuntimeException("Valor das parcelas diferente do
valor total de produtos, verifique");

                Optional<Caixa> caixa = caixas.caixaAberto();
```

```
                Aplicacao aplicacao = Aplicacao.getInstancia();
                Usuario usuario = usuarios.buscaUsuario(aplicacao.getUsuarioAtual());

                Double valor_parcela = (Double.valueOf(vlParcelas[i]) + acre) - desc;
                CaixaLancamento lancamento = new CaixaLancamento("Recebimento de
venda á vista", valor_parcela,
                                TipoLancamento.RECEBIMENTO, EstiloLancamento.ENTRADA,
caixa.get(), usuario);

                try {
                        lancamentos.lancamento(lancamento);
                } catch (Exception e) {
                        System.out.println(e);
                        throw new RuntimeException("Erro ao fechar a venda, chame o
suporte");
                }
                return qtdVezes;
        }

        private Boolean vendaIsAberta(Long codVenda) {
                Venda venda = vendas.findByCodigoEquals(codVenda);
                return venda.isAberta();
        }

        public int qtdAbertos() {
                return vendas.qtdVendasEmAberto();
        }

}
```

Created with JaCoCo 0.8.11.202310140853